# CHAPTER 4: SYSTEM DESIGN

## 4.1 Introduction

This chapter presents the detailed system design of the **Biomech-Coach** application. It details the structural architecture, the finite-state machine (FSM) transitions, the local database schema, and the user interface design. The design is engineered to support low-latency on-device processing and robust offline storage.

## 4.2 System Architecture

The application is structured using a modular architecture based on the **Model-View-Controller (MVC)** design pattern, modified to integrate localized machine learning and biomechanical analysis services. This division separates camera capture and UI rendering from joint angle analysis and state transitions.

The data flow and interactions between modules are structured as follows:

1. **View Layer:** Comprises Flutter screens and painter overlays. The live camera preview captures raw video frames, and a custom skeleton overlay paints coordinates directly onto the screen.
2. **Controller Layer:** Managed by the camera controller and State Providers. It coordinates the lifecycle of the camera stream and routes landmark data to the analytical services.
3. **Model Layer:** Defined by Hive database models (LiftSession and RepRecord), which represent local persistent structures.
4. **Service & Engine Layer:** Houses the core processing modules, including Google ML Kit Pose Detection, the Biomechanics Engine, the Repetition State Machine, the Text-to-Speech (TTS) Coach, and the Backup Service.

This structural data flow is illustrated in Figure 4.1.




## 4.3 Biomechanical State Machine Design

Repetition validation is executed via a deterministic Finite State Machine (FSM). The FSM evaluates joint coordinate angles sequentially to transition between movement phases. This prevents half-reps or poor-form movements from being recorded.

### 4.3.1 Repetition States

The lifecycle of a single repetition transitions through the following states:

- idle: The athlete is standing or holding the starting position. The joint angles are above the start threshold.
- descending: The athlete begins the eccentric phase of the lift. Joint angles decrease.
- atDepth: The target depth is achieved (e.g., hip crease below knee level for a squat).
- ascending: The athlete executes the concentric phase, pushing upward. Joint angles increase.
- lockout: The joint angles reach full extension, completing the range of motion.
- complete: The repetition is verified as valid or invalid, logged to the database, and the state resets back to idle.

The state transition path is mapped in Figure 4.2.

![Figure 4.2: Repetition State Machine Transitions](diagrams/state_machine.png)

### 4.3.2 Lift Transition Parameters

The state transitions are governed by angle thresholds configured in lift_thresholds.dart. Table 4.1 summarizes the exact joint parameters used to evaluate each lift.

**Table 4.1: State Machine Transition Constraints**

| Lift Type | Primary Joint (Depth) | Start Descent Angle | Target Depth Angle | Lockout Joint & Angle |
|---|---|---|---|---|
| **Squat** | Hip Flexion (Shoulder-Hip-Knee) | < 138.0° | ≤ 90.0° | Hip & Knee ≥ 145.0° |
| **Bench Press** | Elbow Flexion (Shoulder-Elbow-Wrist) | < 135.0° | ≤ 90.0° | Elbow ≥ 140.0° |
| **Deadlift** | Hip Flexion | < 138.0° | ≤ 75.0° | Hip & Knee ≥ |

| | | | | |
|---|---|---|---|---|
| | (Shoulder-Hip-Knee) | | | 145.0° |

---

### 4.4 Database Schema Design

For persistent storage, the app utilizes **Hive**, a lightweight key-value database. It runs in pure Dart and saves objects directly in binary format to local memory.

#### 4.4.1 Entity Relationship

The data model consists of two main classes registered with TypeAdapters:

1. **`LiftSession` (TypeID: 0):** Stores metadata of the workout session. It has a one-to-many relationship with RepRecord.
2. **`RepRecord` (TypeID: 1):** Stores the telemetry and results of individual repetitions.

#### 4.4.2 Schema Details

The properties of the database entities are structured as follows:

```dart
// RepRecord Schema (TypeID: 1)
class RepRecord {
DateTime timestamp;      // Exact completion time
bool isValid;            // True if no faults were detected
double formScore;        // Calibrated score out of 100
double minDepthAngle;    // Lowest angle reached at bottom
double lockoutAngle;     // Angle reached at lockout
List<String> faultNotes; // List of detected faults
double durationSeconds;  // Time taken to complete the rep
}
// LiftSession Schema (TypeID: 0)
```

```
class LiftSession {
DateTime startTime;      // Session start timestamp
String liftType;         // 'squat', 'benchPress', or 'deadlift'
List<RepRecord> reps;    // Collection of completed repetitions
DateTime endTime;        // Session end timestamp
}
```

### 4.4.3 Backup File Format Schema

The backup service serializes the collection of LiftSession objects into a single portable JSON file. The JSON schema is structured as a list of session objects containing nested lists of repetition records:

```
`json
[
{
"startTime": "2026-06-15T10:00:00.000Z",
"liftType": "squat",
"endTime": "2026-06-15T10:15:00.000Z",
"reps": [
{
"timestamp": "2026-06-15T10:01:30.000Z",
"isValid": true,
"formScore": 100.0,
"minDepthAngle": 82.5,
"lockoutAngle": 146.2,
"faultNotes": [],
```

"durationSeconds": 2.4

}

]

}

]

---

### 4.5 User Interface Design

The user interface is designed using a clean, modern HUD aesthetic with frosted glass elements. The layout consists of four primary screens:

1. **Dashboard (Home Screen):** Contains exercise selection cards (Squat, Bench Press, Deadlift). It displays overall weekly volume and summary stats.
2. **Camera HUD Screen:** Displays the live camera view with a overlaying color-coded skeletal frame. It features floating real-time angle gauges on the left and a prominent rep counter on the right.
3. **Session Summary Screen:** Appears when a workout is ended. It displays the average form score, valid vs. invalid rep ratio, and historical trend lines.
4. **Journal Screen:** Lists historical sessions sequentially. It contains **Import** and **Export** buttons in the top header, allowing users to select or share backup files locally.